

Introduction to Julia : GPU programming

Matteo Foglieni, Dr. Martin Ohlerich



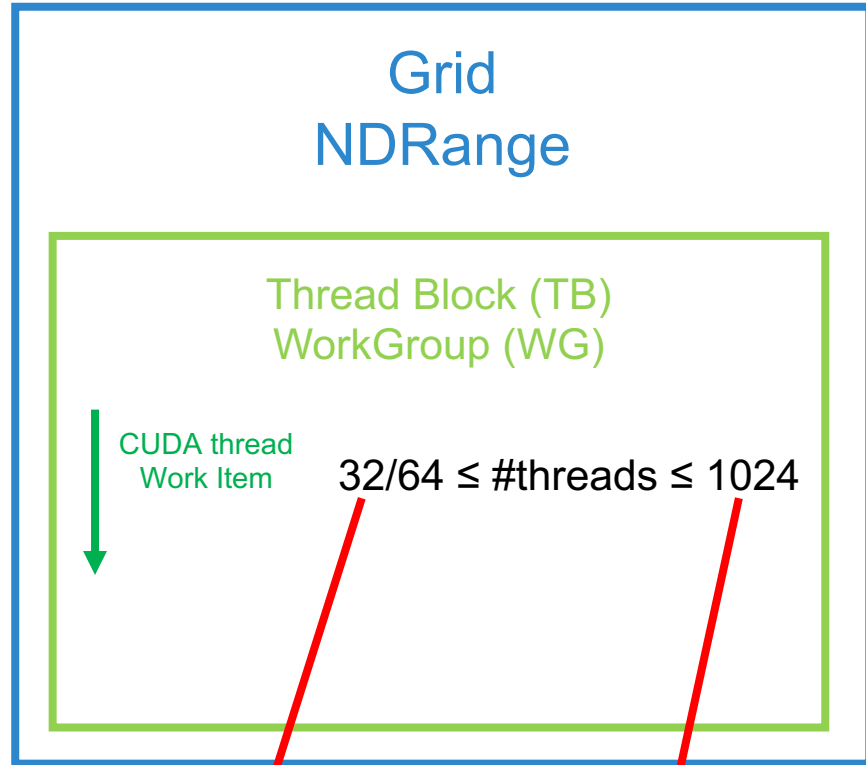
GPU: NVIDIA A100 (GA100)

- └ GPC ×7 Graphics Processing Cluster
- └ TPC ×7-8/GPC Texture Processing Cluster
- └ SM ×2/TPC, 108 total **Streaming Multiprocessor (SMs)**
 - └ Sub-partition ×4
 - └ Warp Scheduler, 32 threads/clock
 - └ Dispatch Unit, 32 threads/clock
 - └ Register File, 16,384 × 32-bit
 - └ **FP32** ×16/sub -> **64/SM**, aka **"CUDA cores" or "Shading Units"**
 - └ INT32 ×16/sub -> 64/SM
 - └ FP64 ×8/sub -> 32/SM, 1:2 FP64:FP32, HPC grade
 - └ **Tensor Core** ×1/sub -> **4/SM**
 - └ LD/ST ×8/sub -> 32/SM, load/store units
 - └ SFU ×1/sub -> 4/SM, sin/cos/exp/rsqrt
 - └ L1 Data Cache / Shared Memory 192KB/SM
 - └ Tex (TMU) ×4 -> 4/SM

Total: **6912 Shading Units, 432 Tensor Cores**

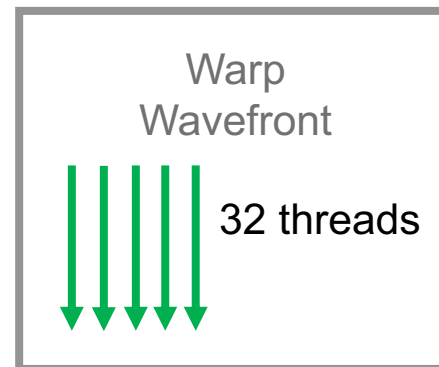
Sources: [NVIDIA A100 Tensor Core GPU Architecture](#), page 22, and [TechPowerUp page for A100 GPU](#)

PROGRAM ABSTRACTIONS

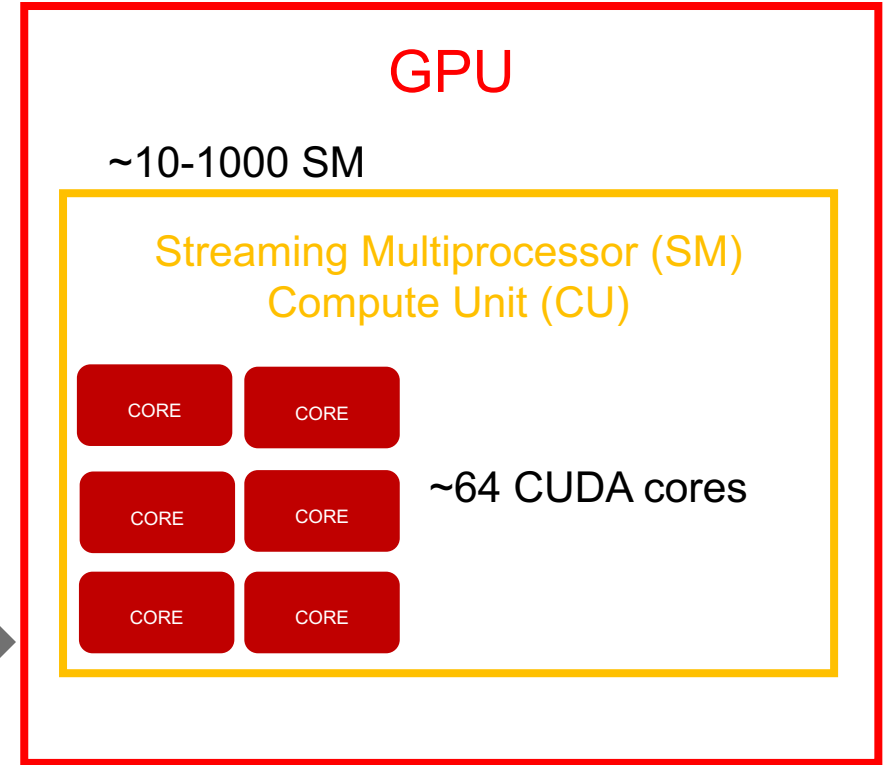


TO BE CHOSEN:
#Thread Blocks
#threads in each TB

one core executes
instructions from one thread



GPU HARDWARE



practical efficiency minimum
(= 1 warp), not a constraint

warp scheduler **hardware constraint**:
it can track at most 32 warps per block
each warp is 32 threads
 $32 \times 32 = 1024$

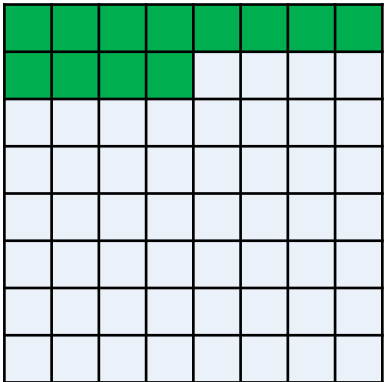
typical unit of execution on a GPU
all threads in a warp are scheduled together and
execute in parallel onto a single SM

E.g.: #threads in each Thread Block = 64 = 8x8

E.g.: Vector of length 12



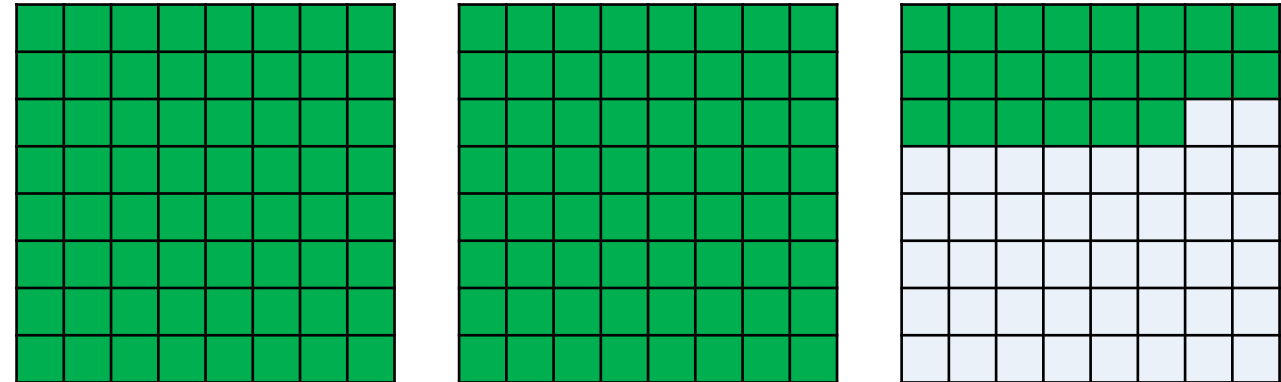
1 TB is enough



E.g.: Vector of length 150



We need 3 TBs (at least)



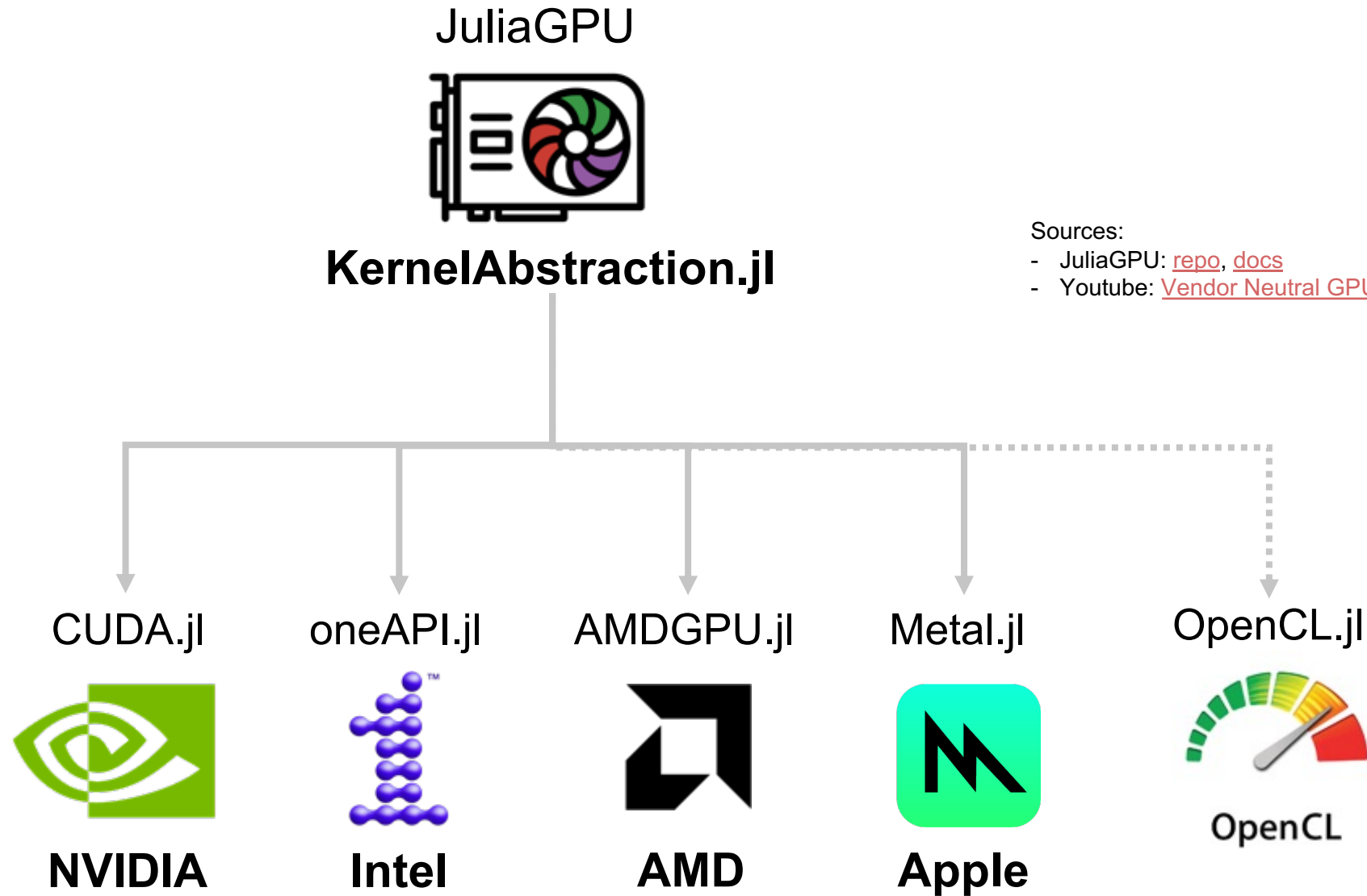
64

+

64

+

22



KernelAbstractions.jl

KA example

```
1 using KernelAbstractions, Random, Test
2 using Metal
3 const DevLibrary = Metal
4 const backend = Metal.MetalBackend()
5 const DevArray = Metal.MtlArray
6 const DevFloat = Float32
```

only part
to be customized!

```
8 @kernel function add_arrays!(a, b, c)
9     i = @index(Global)
10    c[i] = a[i] + b[i]
11 end
```

```
12
13 N = 1024
14 dev_a = rand!(allocate(backend, DevFloat, N))
15 dev_b = rand!(allocate(backend, DevFloat, N))
16 dev_c = similar(dev_a)
```

julia> include("../KA-outline_Metal.jl")
Test Passed

```
17
18 # Compile the kernel for this backend
19 compiled_kernel! = add_arrays!(backend, 64)
20 # Launch the kernel
21 event = compiled_kernel!(dev_a, dev_b, dev_c; ndrange=size(dev_a))
22 KernelAbstractions.synchronize(backend)
23
24 a, b, c = Array(dev_a), Array(dev_b), Array(dev_c)
25 @test all([z ≈ x + y for (x,y,z) in zip(a,b,c)])
```

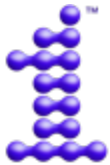
WorkGroup (Thread Block) size; use 32 or 64

NDRange (Grid) size; set to vector size

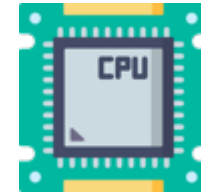
Backends



```
2 using CUDA
3 const DevLibrary = CUDA
4 const backend = CUDA.CUDAKernels.CUDABackend()
5 const DevArray = CUDA.CuArray
6 const DevFloat = Float64
```



```
2 using oneAPI
3 const DevLibrary = oneAPI
4 const backend = oneAPI.oneAPIKernels.oneAPIBackend()
5 const DevArray = oneAPI.OneArray
6 const DevFloat = Float64
```



```
2 const backend = CPU()
3 const DevArray = Array
4 const DevFloat = Float64
```



```
2 using AMDGPU
3 const DevLibrary = AMDGPU
4 const backend = AMDGPU.ROCKernels.ROCBackend()
5 const DevArray = AMDGPU.ROCArray
6 const DevFloat = Float64
```



```
2 using Metal
3 const DevLibrary = Metal
4 const backend = Metal.MetalBackend()
5 const DevArray = Metal.MtlArray
6 const DevFloat = Float32
```


Backend.jl

Why not using

```
if Base.find_package("oneAPI") != nothing
```

?



We can unify the previous code snippets into a single Backend.jl file!

Portability ✓

```
1 using Adapt, KernelAbstractions
2
3 onlycpu = (@isdefined onlycpu) ? onlycpu : false
4 if Base.find_package("oneAPI") != nothing && onlycpu == false
5     using oneAPI
6     const backend = oneAPI.oneAPIKernels.oneAPIBackend()
7     const DevArray = oneAPI.oneArray
8     const DevFloat = Float64
9     println("Using oneAPI.jl")
10    oneAPI.versioninfo()
11
12 elseif Base.find_package("CUDA") != nothing && onlycpu == false
13     using CUDA
14     const backend = CUDA.CUDAKernels.CUDABackend()
15     const DevArray = CuArray
16     const DevFloat = Float64
17     CUDA.allowscalar(false)
18     println("Using CUDA.jl")
19     CUDA.versioninfo()
20
21 elseif Base.find_package("AMDGPU") != nothing && onlycpu == false
22     using AMDGPU
23     const backend = AMDGPU.ROCKernels.ROCBackend()
24     const DevArray = ROCArray
25     const DevFloat = Float64
26     println("Using AMDGPU.jl")
27     AMDGPU.versioninfo()
28
29 elseif Base.find_package("Metal") != nothing && onlycpu == false
30     using Metal
31     const backend = Metal.MetalKernels.MetalBackend()
32     const DevArray = Metal.MtlArray
33     const DevFloat = Float32
34     println("Using Metal.jl => Float32")
35     Metal.versioninfo()
36
37 else
38     using ThreadPinning
39     const backend = CPU()
40     const DevArray = Array
41     const DevFloat = Float64
42     println("Using CPU and ThreadPinning.jl")
43     pinthreads(:cores)
44     threadinfo()
45 end
```

Common API of Backends



Example: use versioninfo()

Reproducibility 

```
julia> versioninfo()
Julia Version 1.10.9
Commit 5595d20a287 (2025-03-10 12:51 UTC)
Build Info:
  Official https://julialang.org/ release
Platform Info:
  OS: macOS (arm64-apple-darwin24.0.0)
  CPU: 10 × Apple M1 Pro
  WORD_SIZE: 64
  LIBM: libopenlibm
  LLVM: libLLVM-15.0.7 (ORCJIT, apple-m1)
Threads: 4 default, 0 interactive, 2 GC (on 8 virtual cores)
Environment:
  JULIA_NUM_THREADS = 4
```

```
julia> oneAPI.versioninfo()
Binary dependencies:
- NEO: 24.26.30049+0
- libigc: 1.0.17193+0
- gnmplib: 22.3.20+0
- SPIRV_LLVM_Translator: 20.1.0+2
- SPIRV_Tools: 2025.1.0+1

Toolchain:
- Julia: 1.10.9
- LLVM: 15.0.7

1 driver:
- 00000000-0000-0000-1847-b9f301037561 (v1.3.30049, API v1.3.0)

8 devices:
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
```

NOTE: 2 tiles per GPU

Example of Adapt custom structs in KA

Using struct in kernel

We need to import a struct as a params in the kernel

```
1  include("../Backend.jl")
2  using Test
3
4  struct MyStruct
5      v::Vector{Float32}
6      f::Float32
7  end
8
9  MS = MyStruct([1.0f0, 2.0f0], 3.0f0)
10
11  @kernel function simple_kernel!(B, A, @Const(ms))
12      i = @index(Global)
13      B[i] = ms.f * A[i] + ms.v[2]
14  end
15
16  dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512)
17  dev_B = similar(dev_A)
18  compiled_kernel! = simple_kernel!(backend, 64)
19  compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
20  KernelAbstractions.synchronize(backend)
21
22  B = Array(dev_B)
23  @test all(B .≈ DevFloat(5.0))
```



```
julia> include("../GaPSE-KA_error-isbits-1.jl")
Using Metal.jl => Float32
ERROR: LoadError: GPU compilation of MethodInstance for
gpu_simple_kernel! (::KernelAbstractions.CompilerMetadata{...}, ::MtlDeviceMatrix{...}, ::MtlDeviceMatrix{...}, ::MyStruct) failed
KernelError: passing non-bitstype argument

Argument 5 to your kernel function is of type MyStruct,
which is not a bitstype:
    .v is of type Vector{Float32} which is not isbits.
```

Using struct in kernel

maybe if I use the Device Array...



```
1  include("../Backend.jl")
2  using Test
3
4  struct MyStruct
5      v::DevArray{Float32}
6      f::Float32
7  end
8
9  MS = MyStruct(DevArray([1.0f0, 2.0f0]), 3.0f0)
10
11  @kernel function simple_kernel!(B, A, @Const(ms))
12      i = @index(Global)
13      B[i] = ms.f * A[i] + ms.v[2]
14  end
15
16  dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512)
17  dev_B = similar(dev_A)
18  compiled_kernel! = simple_kernel!(backend, 64)
19  compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
20  KernelAbstractions.synchronize(backend)
21
22  B = Array(dev_B)
23  @test all(B .≈ DevFloat(5.0))
```

```
julia> include("../GaPSE-KA_error-isbits-2.jl")
Using Metal.jl => Float32
ERROR: LoadError: GPU compilation of MethodInstance for
gpu_simple_kernel! (::KernelAbstractions.CompilerMetadata{...}, ::MtlDeviceMatrix{...}, ::MtlDeviceMatrix{...}, ::MyStruct) failed
KernelError: passing non-bitstype argument

Argument 5 to your kernel function is of type MyStruct,
which is not a bitstype:
```


Using struct in kernel



```
1 include("../Backend.jl")
2 using Test
3
4 struct MyStruct{V,F}
5     v::V      # Vector{Float32}
6     f::F      # Float32
7 end
8
9 function Adapt.adapt_structure(to, ms::MyStruct)
10     MyStruct(
11         adapt(to, ms.v),
12         ms.f
13     )
14 end
15
16 #Adapt.@adapt_structure MyStruct # easier, but not always possible
17 MS = MyStruct(DevArray([1.0f0, 2.0f0]), 3.0f0)
18
19 @kernel function simple_kernel!(B, A, @Const(ms))
20     i = @index(Global)
21     B[i] = ms.f * A[i] + ms.v[2]
22 end
23
24 dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512) # alloc on device
25 dev_B = similar(dev_A)
26 compiled_kernel! = simple_kernel!(backend, 64)
27 compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
28 KernelAbstractions.synchronize(backend)
29
30 B = Array(dev_B)
31 @test all(B .≈ DevFloat(5.0))
32
```

Parametric Struct + Adapt.jl

https://cuda.juliagpu.org/stable/tutorials/custom_structs

```
julia> include("../GaPSE-KA_working.jl")
Using Metal.jl => Float32
Test Passed
```


For GaPSE.jl we did something more

```

82 struct MySpline
83     xs::Vector{DevFloat}
84     coeffs::Vector{Tuple{DevFloat,DevFloat,DevFloat,DevFloat}}
85     N::Int64
86     function MySpline(xs, ys; bc="Error", ic="Natural")
87         #... # cubic spline computation
88         new(xs, coeffs, N)
89     end
90 end
91
92
93 function (S::MySpline)(x)
94     @assert S.xs[1] ≤ x ≤ S.xs[end] "BC Error"
95     i = searchsortedlast(S.xs, x)
96     u, a, b, c, d = (i == length(S.xs)) ?
97         (x - S.xs[i-1], S.coeffs[i-1]...) :
98         (x - S.xs[i], S.coeffs[i]...)
99     (u ≈ 0.0) && (return S.coeffs[i][1])
100
101     return a + b * u + c * u^2 + d * u^3
102 end
    
```

Original Spline implementation



```

106 struct DevMySpline{V,VT,I}
107     xs::V #Vector{DevFloat}
108     coeffs::VT #Vector{Tuple{DevFloat,DevFloat,DevFloat,DevFloat}}
109     N::I #Int64
110 end
111
112 function Adapt.adapt_structure(to, s::MySpline)
113     return DevMySpline(
114         adapt(to, s.xs),
115         adapt(to, s.coeffs),
116         adapt(to, s.N))
117 end
118
119 function Adapt.adapt_structure(to, s::DevMySpline)
120     DevMySpline(
121         adapt(to, s.xs),
122         adapt(to, s.coeffs),
123         adapt(to, s.N))
124 end
125
126
127 function (S::DevMySpline)(x)
128     i = searchsortedlast(S.xs, x)
129     u, a, b, c, d = (i == length(S.xs)) ?
130         (x - S.xs[i-1], S.coeffs[i-1]...) :
131         (x - S.xs[i], S.coeffs[i]...)
132     (u ≈ zero(u)) && (return S.coeffs[i][1])
133
134     return a + b * u + c * u^2 + d * u^3
135 end
    
```

Code added

Example of usage of a MySpline in a kernel:

```
138 xs = [1.0f0 * i for i in 1:20]
139 ys = xs.^2
140 MS = MySpline(xs, ys)
141
142 xs_dev = DevArray(xs)
143 ys_dev = DevArray(ys)
144 MS_dev = adapt(backend, MS)
145
146 @kernel function spline_kernel!(b, a, @Const(ms))
147     i = @index(Global, Linear)
148     b[i] = ms(a[i])
149 end
150
151 a = [DevFloat(MS.xs[begin] + rand() * (MS.xs[end] - MS.xs[begin])) for i in 1:1024]
152 a_dev = DevArray(a)
153 b_dev = similar(a_dev)
154
155 spline_kernel!(backend, 64)(b_dev, a_dev, MS_dev, ndrange=size(a_dev))
156 KernelAbstractions.synchronize(backend)
157
158 b = Array(b_dev)
159
160 @test all(b.≈MS.(a))
```

} conversion with Adapt.jl

```
julia> include("../GaPSE-KA_devstruct.jl")
Using Metal.jl => Float32
Test Passed
```

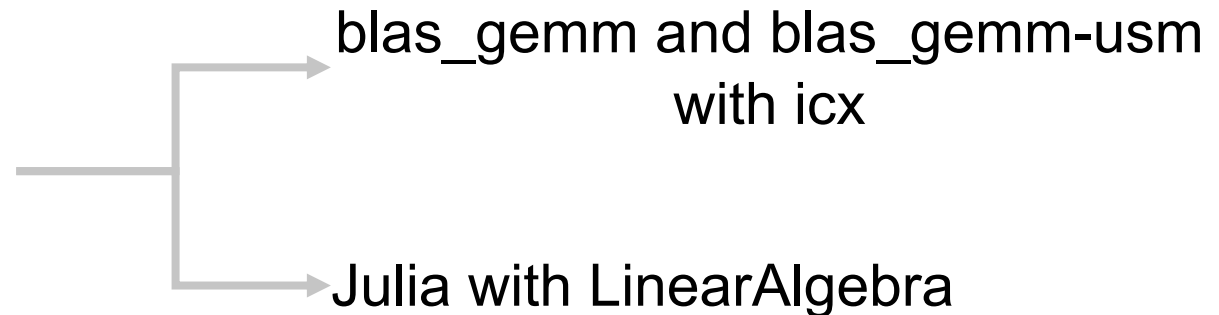
Appendices

Julia vs C++ GEMM on LRZ hardware (CPU only)

GEMM benchmarking comparison on LRZ Hardware



we chose to compare
a simple matrix multiplication



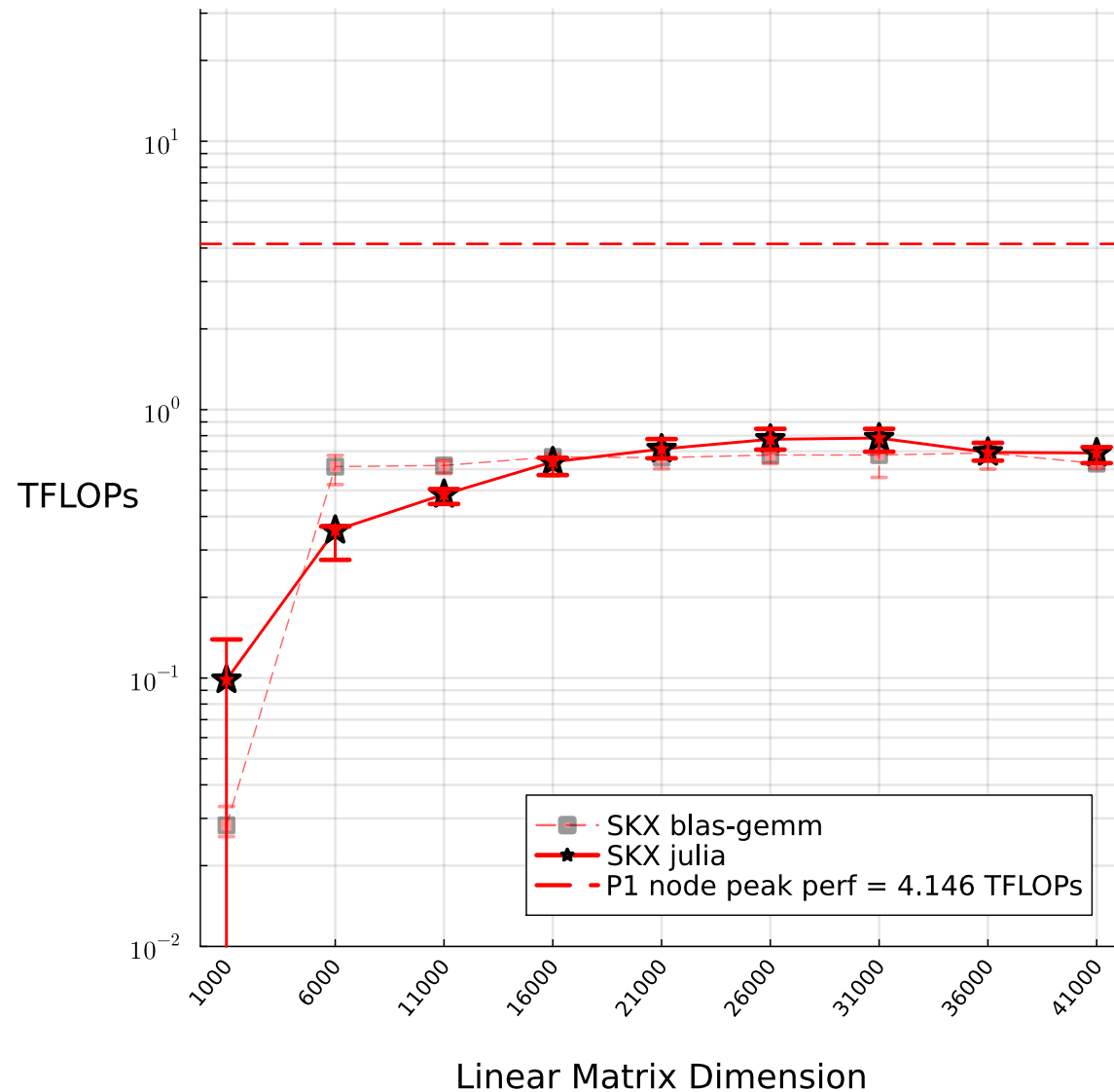
$$R = \alpha \overset{M \times N}{\boxed{A}} * \overset{N \times K}{\boxed{B}} + \beta \overset{M \times K}{\boxed{C}} \quad \alpha, \beta \in \mathbb{R}$$

MLD = Matrix Linear Dimension

Our simplification: $M = N = K = \mathbf{MLD}$ $\alpha = 1, \beta = 0$

$$\mathbf{FLOPs} = \frac{\# \text{FP ops}}{\text{time}} \approx \frac{M \times N \times K}{\Delta t} = \frac{\mathbf{MLD}^3}{\Delta t}$$

GEMM benchmarking comparison on LRZ hardware



SKX = Intel “Skylane” Xeon Platinum 8174

SPR = Intel “Sapphire Rapid” Xeon Platinum 8480L

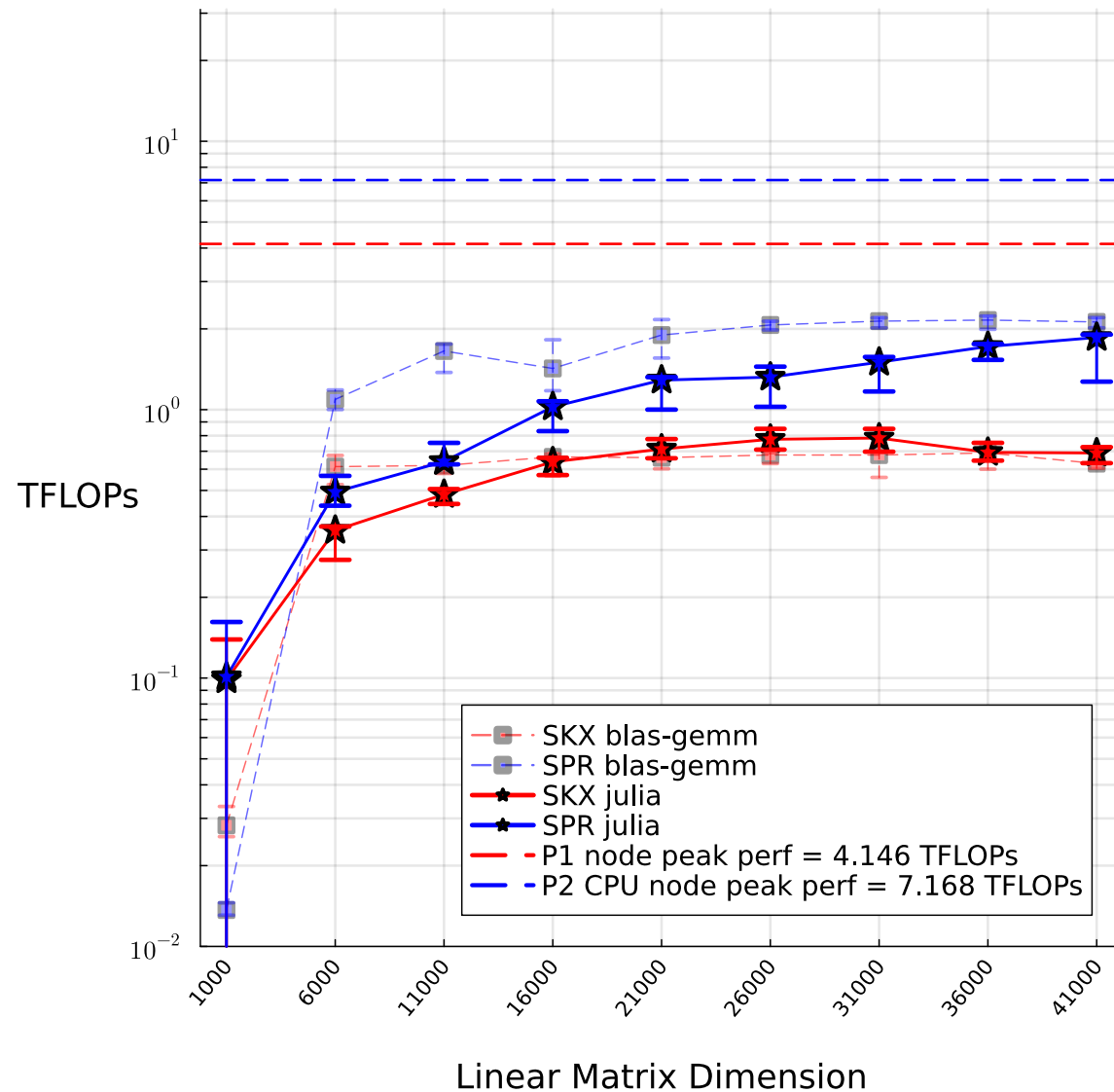
PVC = Intel “Ponte Vecchio” Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

↑ higher is better

GEMM benchmarking comparison on LRZ hardware



SKX = Intel "Skylane" Xeon Platinum 8174

SPR = Intel "Sapphire Rapid" Xeon Platinum 8480L

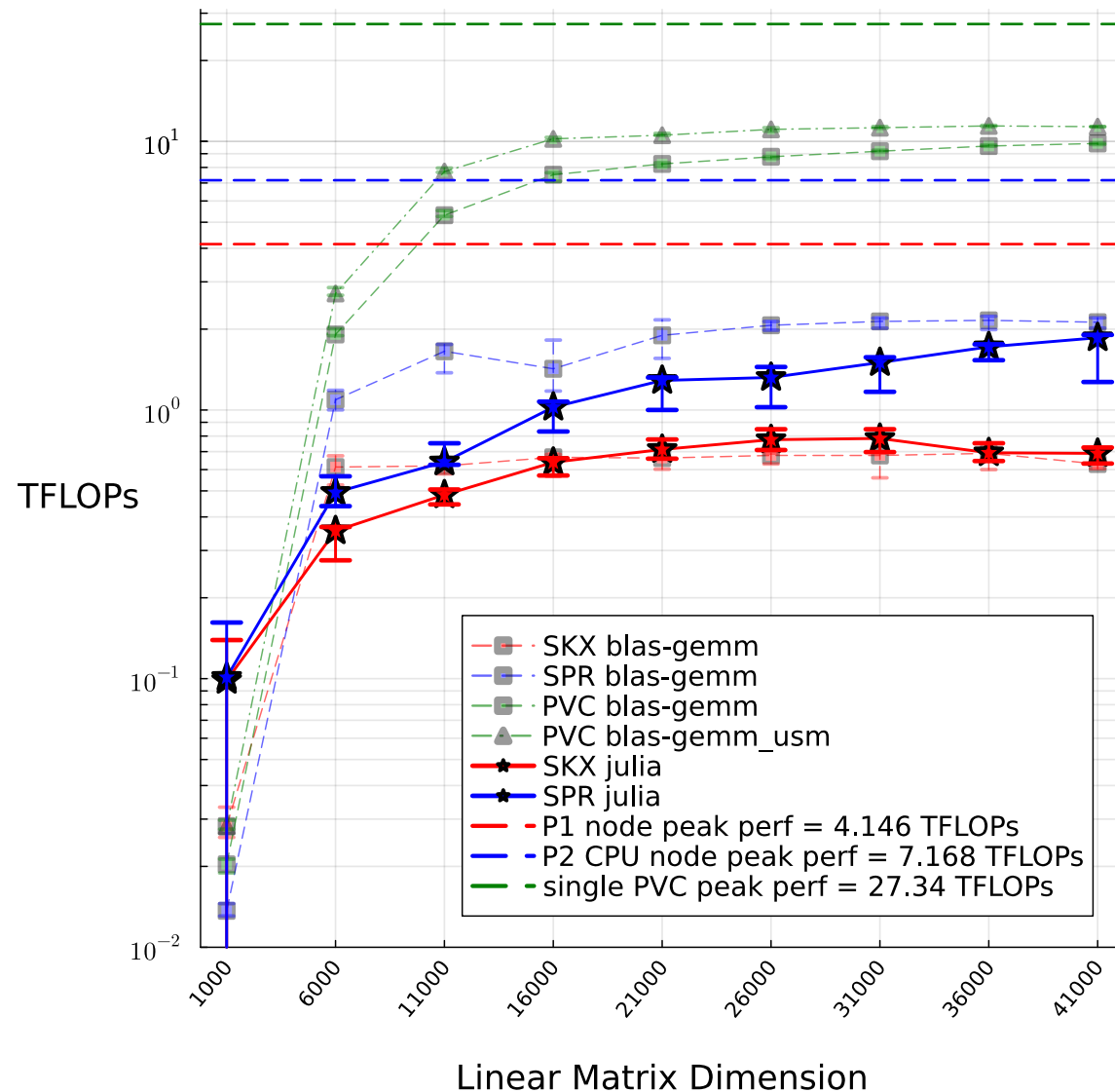
PVC = Intel "Ponte Vecchio" Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

↑ higher is better

GEMM benchmarking comparison on LRZ hardware



SKX = Intel "Skylane" Xeon Platinum 8174

SPR = Intel "Sapphire Rapid" Xeon Platinum 8480L

PVC = Intel "Ponte Vecchio" Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

↑ higher is better